

בדיקת מערכות ספרתיות ואימות תכנון

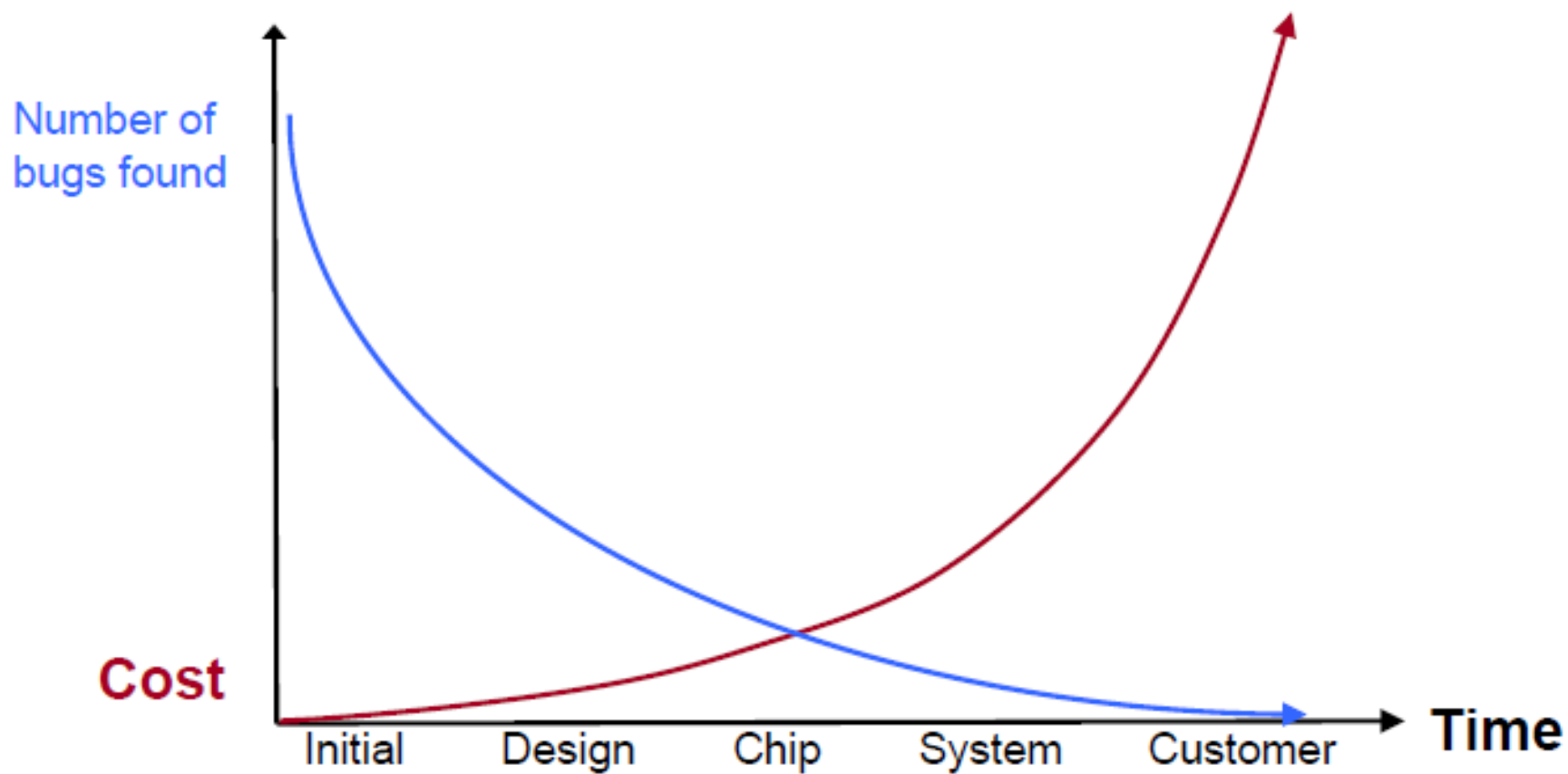
פרק 3 : סימולציה

חלק 1

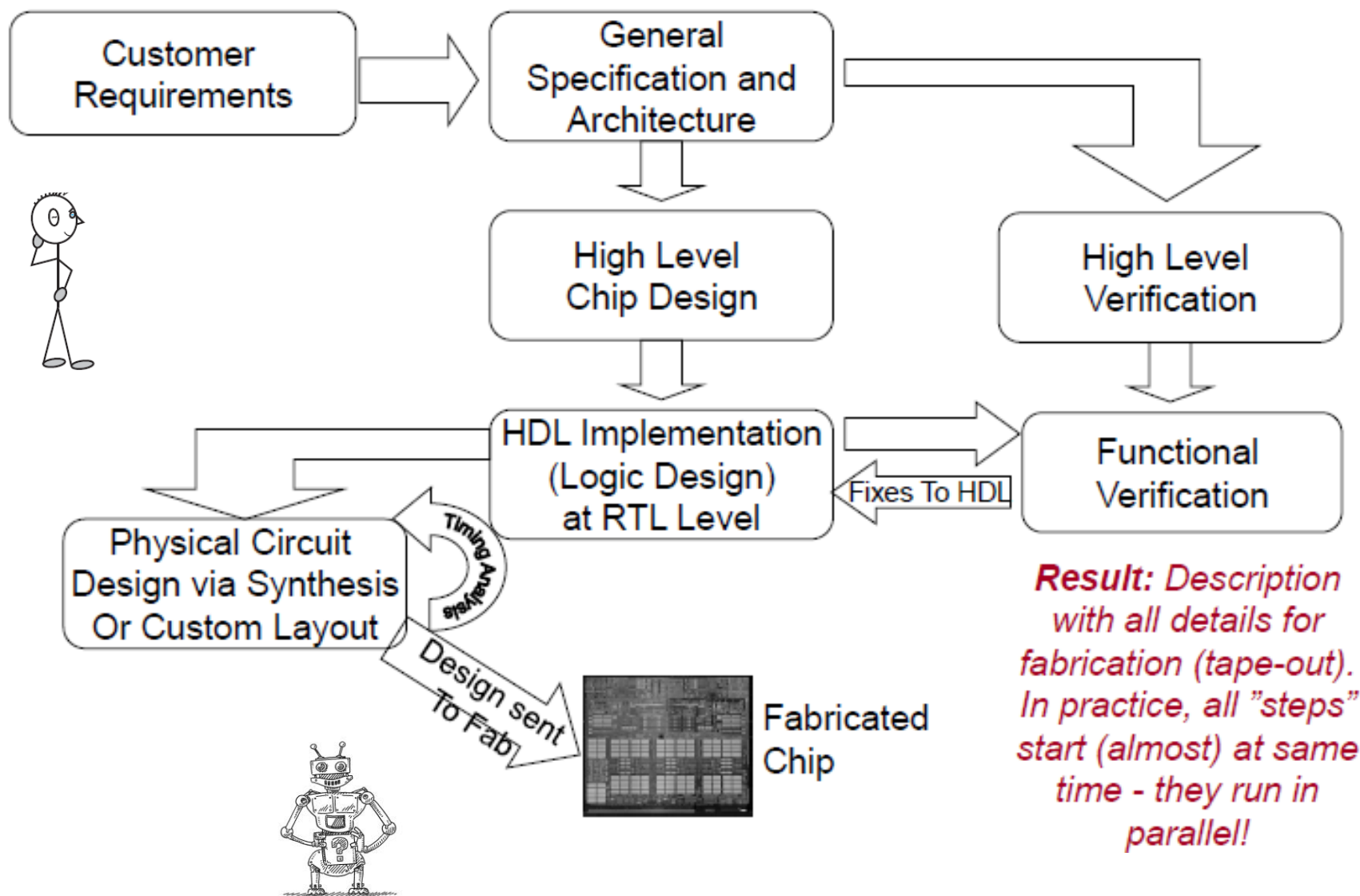
מה ואיך בודקים



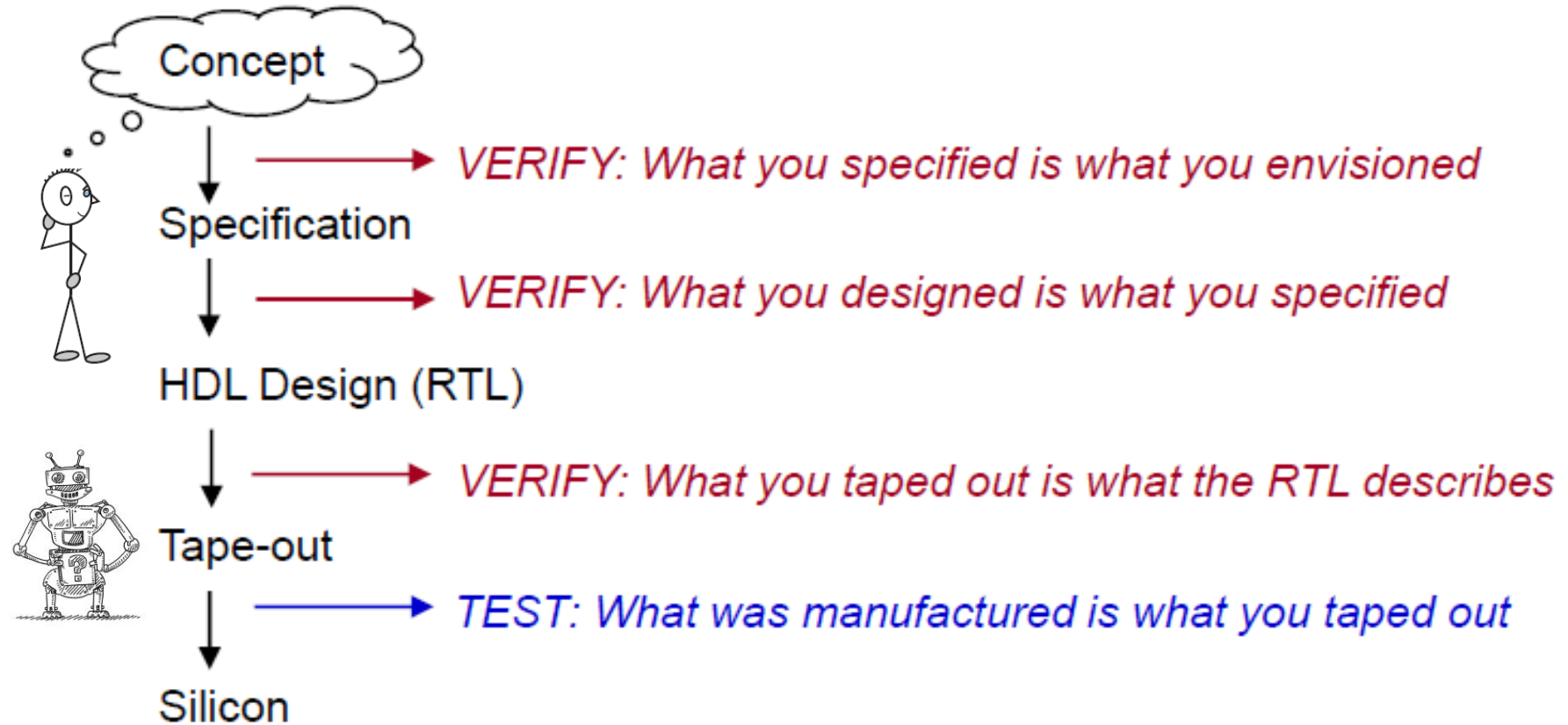
עלות של שגיאות לאורך זמן



תהליך תכנון ספרתי



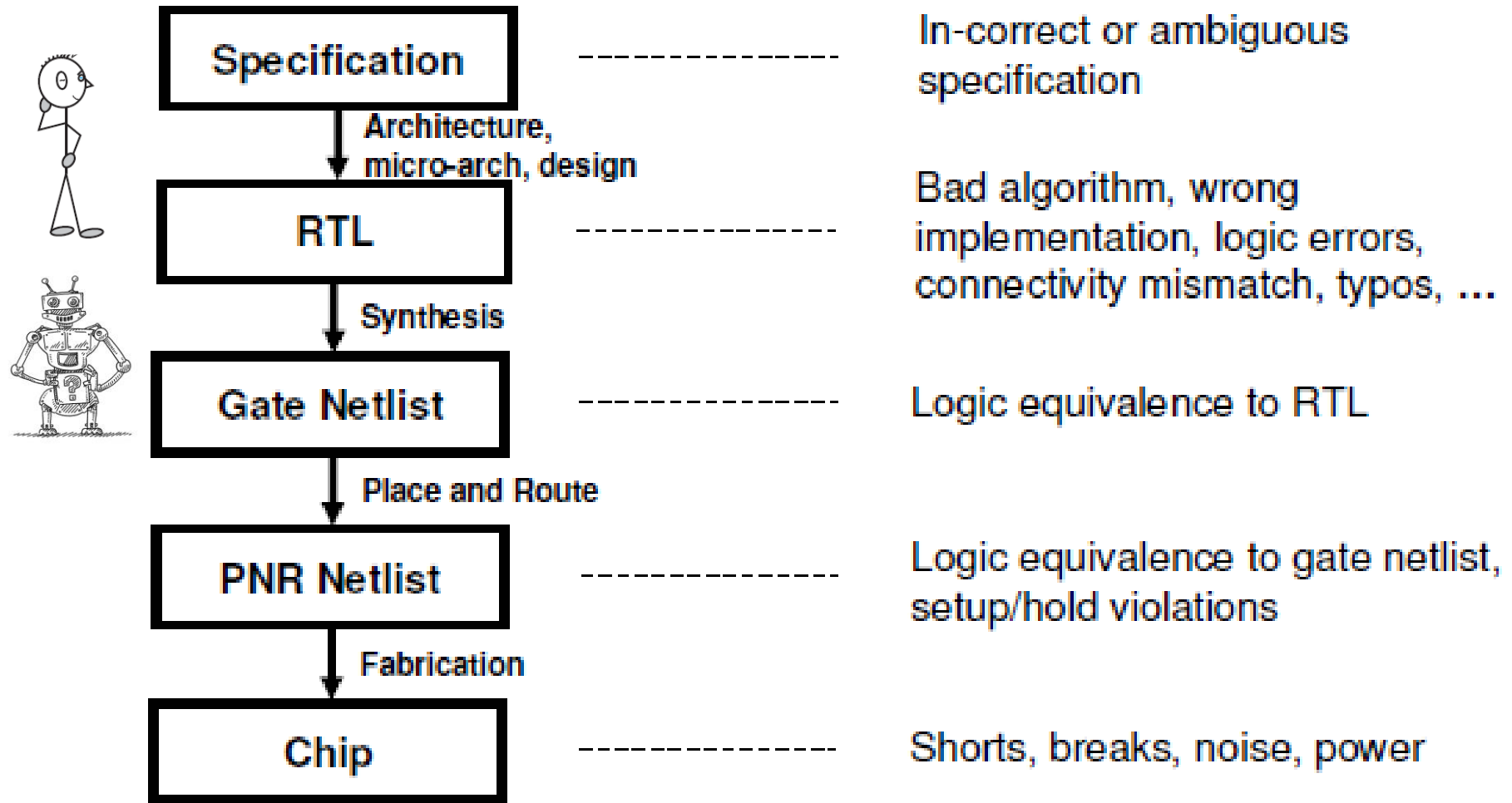
איך מהנדס יודע שהתכנון נכון?



איפה דברים עלולים להשתבש? (בכל מקום...)

Chip Design Process

Typical Issues



סימולציות ברמות שונות

- SYSTEM SPECIFICATION (*Functional Simulation*)
- ARCHITECTURAL DESIGN (*Block level Simulation*)
- LOGIC DESIGN (*Logic Simulation*)
- CIRCUIT DESIGN (*Circuit Simulation*)
- DEVICE DESIGN (*Device Simulation*)
- LAYOUT
- FABRICATION (*Process Simulation*)
- ON-WAFER TESTING (*Fault Simulation*)
- PACKAGING
- CHIP TESTING

סוגי אימות

Verification



Developer

Verification Agent

Validation



Functional Expert

Validation Agent

Accreditation



Requester/User

Accreditation Agent

As design matures, re-examine basic assumptions.

Verification

אימות זה תהליך המשמש כדי להוכיח את נכונות התכנון
לדרישות המפרט .



סוגי האימות:

1. האימות הפונקציונלי
(COMS31700)
2. אימות תזמון
3. אימות ביצועים

טכניקות אימות פונקציונלית



➤ אימות פורמלית

➤ אימות סטטית

➤ הוכחה מתמטית של נכונות התכנון

■ שיטות מבוססות סימולציה

➤ אימות דינמית

➤ מציאת באגים על ידי הפעלת מודל התכנון ובדיקת
ההתנהגות שלו

❖ שימוש בטסטים לביצוע של הריצות

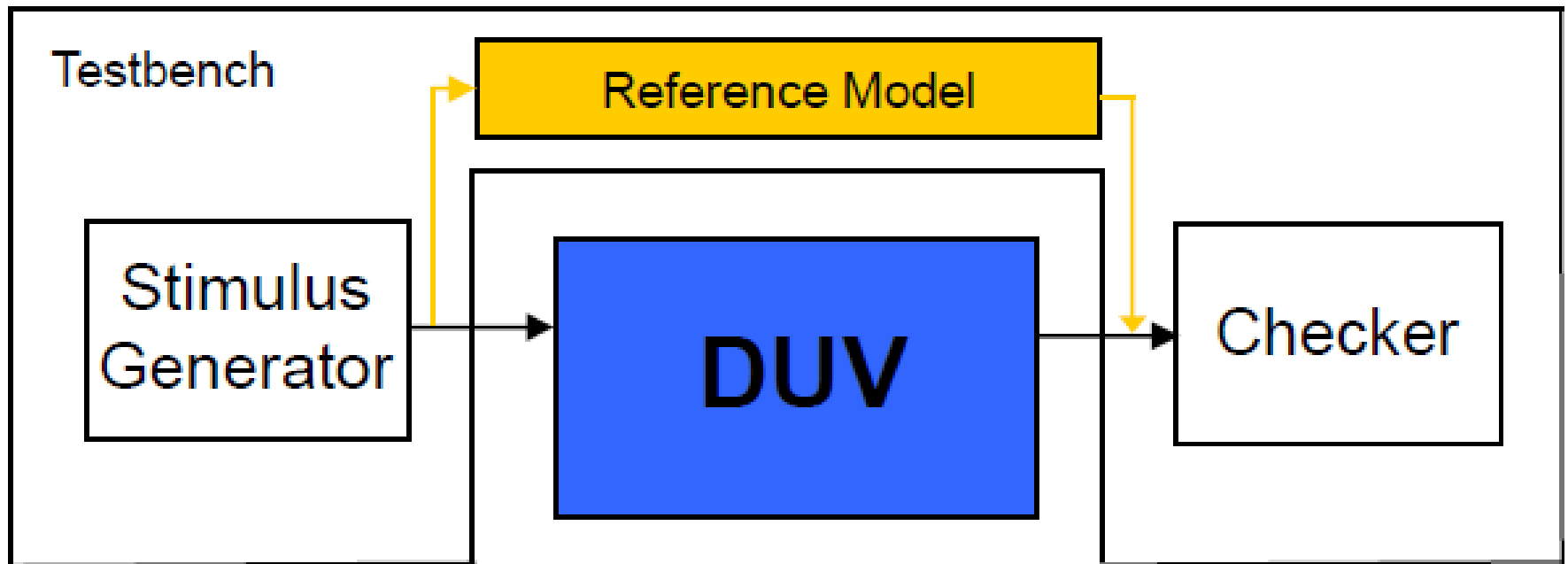
מה זה טסט?

➤ קוד שמשמש כדי לייצר מראש רצף קלט לריצה, וכדי לבחון את התגובה.

➤ באופן מסורתי, טסט מתייחס לקוד שנכתב בשפה לתיאור חומרה (VHDL, Verilog) ברמות שונות של ההיררכיה התכנון.

➤ טסטים כתובים ב-VHDL יחסית פשוטים, אבל כוללים אלמנטים של אקראיות.

מבנה הטסט



התכנסות

ייצוג קונספטואלי של תהליך האימות

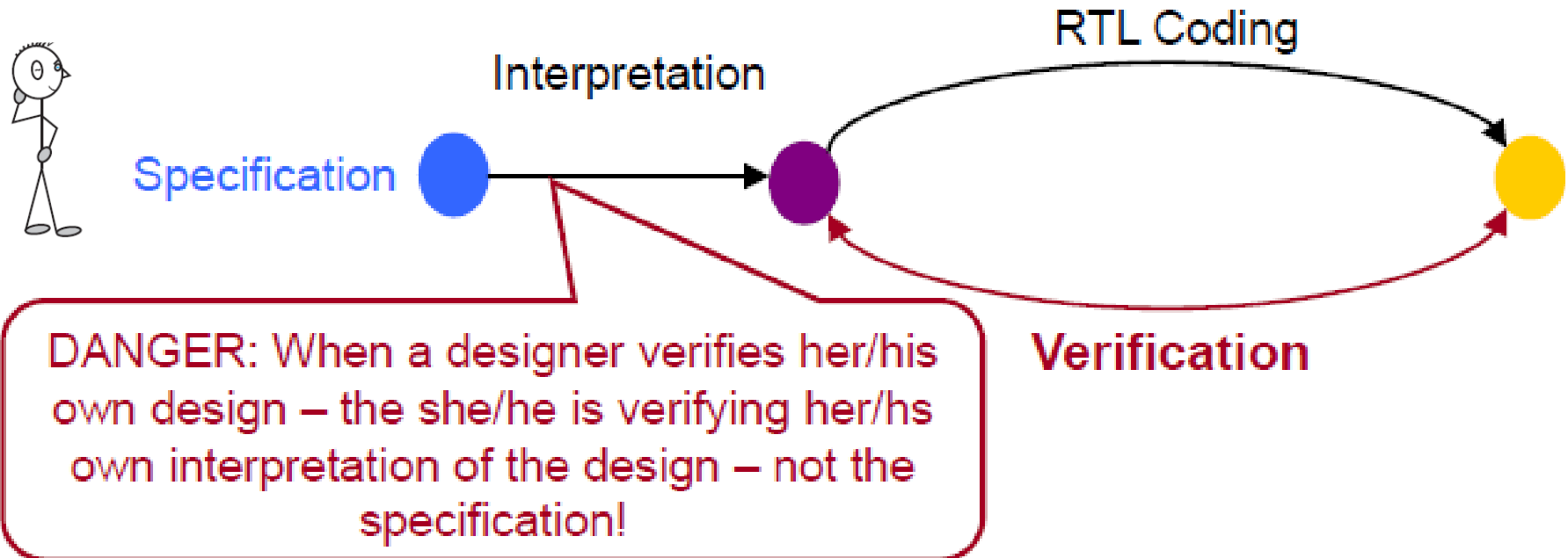


מטרת האימות היא להבטיח כי התוצאה של שינוי כלשהו של
התכנון תהיה כמו שנועד או כמו שצפוי

הגורם האנושי של אימות

אדם (או קבוצה של אנשים) חייב לקרוא ולפרש את המפרט ולהפוך אותו לתכנון.

- בפועל, המפרט הוא בדרך כלל מסמך כתוב בשפה טבעית על ידי אנשים בדרגות שונות של היכולת לתקשר.
- טעויות אנוש נכנסות בעקבות פרשנות מוטעה.



צמצום שגיאות המוכנסות על ידי אנשים



אוטומציה: תוציא את האדם!

➤ בפועל לעתים קרובות לא ריאלי

➤ התהליך לא מוגדר מספיק טוב

(לא פורמאלי מספיק)

➤ טרנספורמציה דורשת תחכום ויצירתיות

הפוך את התערבות אדם ל-"ללא טעות: הוכחה"

➤ בדומה ל-USB

➤ החסרונות כמו באוטומציה: אימות היא אמנות!

צמצום שגיאות המוכנסות על ידי אנשים

יתירות: שכפל כל שינוי

➤ מגייסים שני אנשים (או קבוצות) לבדוק את עבודתם זה של זה

➤ שתי טרנספורמציות נפרדות לחלוטין מתבצעות וכל תוצאה נבדקת לעומת השניה לוודא כי הן זהות או דומות

➤ יקר מועד, אבל עדיין זול יותר מאשר מחדש את התכנון או להחזיר את המוצר

➤ הזהרה: מהנדס אלקטרוניקה לא צריך להיות אחראי על אימות!

מה ואיך בודקים?

לבחור מוצא משותף ולקבוע נקודות התכנסות

מה אמור להיות מאומת?

איזה סוג של שיטה היא הטובה ביותר לשימוש?

לחקור את המודלים של ההתכנסות עבור

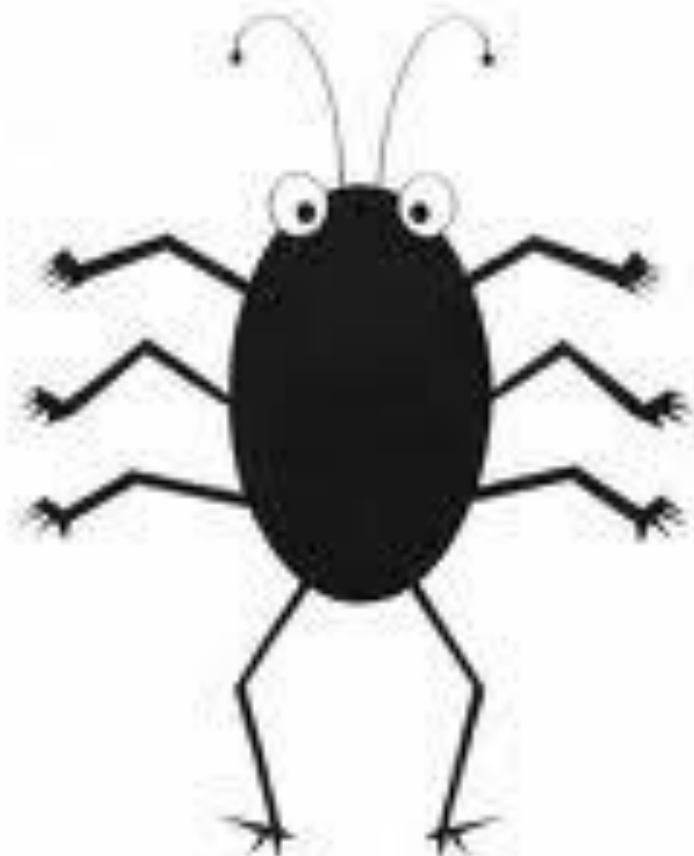
אימות פורמלי

אימות מבוססות סימולציה

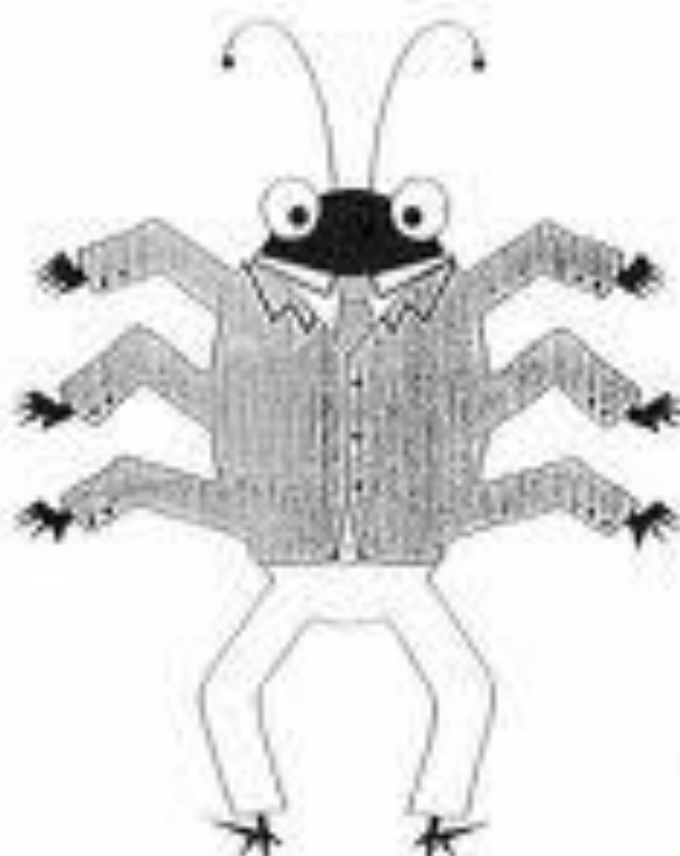
This Image Scan is
© Royce Art

This image belongs to www.magepuzzles.com/royce

מה ואיך בודקים?



BUG



FEATURE

אימות



בדיקה מבוססת סימולציה



➤ האימות הפונקציונלי מאמתת את הכוונה

➤ אם המפרט אינו כתוב עם סמנטיקה מדויקת, אי אפשר להראות את התכנון מקיים את הכוונה של המפרט שלו.

➤ סימולציה יכולה (רק) להראות את נוכחותם של באגים, אבל (בפועל) לא ניתן להוכיח היעדרם!

➤ אמון בסימולציה ככלי לבדיקות הולך ופוחת!

בדיקה מבוססת סימולציה



שקפות במהלך אימות



1. אימות קופסה שחורה -- Black box

2. אימות קופסה לבנה -- White box

3. אימות קופסה אפורה -- Grey box

אימות קופסה שחורה

- לקופסה השחורה יש קלט, פלט, והיא מבצעת כמה הפונקציות (מתועדת היטב).
- כדי לבדוק קופסה שחורה, צריך להבין את התפקיד שלה ולהיות מסוגל לחזות את הפלט על בסיס קלט.
- קוד אימות מנצל רק ממשקים חיצוניים כפי שהוגדר על ידי המפרט. אותות פנימיים נשארים בחושך....



אימות קופסה שחורה



יתרונות:

➤ אין צורך בידע בפועל על היישום

➤ היכולת לחזות תוצאות פונקציונאליות

המבוססות על הקלט לבד ומבטיח כי reference model
נותר עצמאי ולא תלוי ביישום ה-DUV

➤ קוד האימות פחות רגיש לשינויים בתוך DUV

חסרונות:

➤ קשה לאתר את מקור הבעיה, רק חושף תוצאות (אם
בכלל - לא כל הבאגים מתגלגלים עד הפלט).

➤ לא שקופה

אימות מושלמת

כדי לבדוק באופן מלא קופסה שחורה, חייבים להראות כי המוצר פועל כראוי עבור כל הצירופים של הקלט.



- ריצות על כל קלט חוקי
- בדיקות תוצאות בכל המקרים
- אפשרי לאימות קלט של שער AND
- אימות שלמה אינה מעשית על מוצר גדול

אימות קופסה לבנה

➤ עבור אימות קופסה לבנה מבנה הפנימי של DUV ידוע, גלוי ומנוצל לצורך אימות



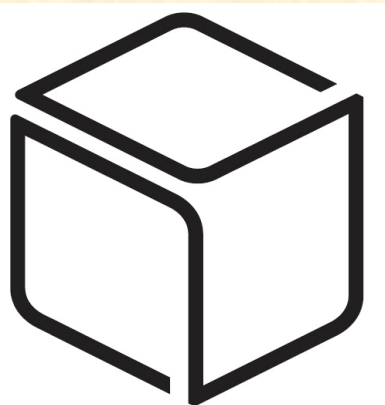
אימות קופסה לבנה

■ יתרונות:

➤ ניתנת לבקרה ושקיפות שלמה של אותות פנימיים

➤ ניתן לזהות באגים ברגע שהם מתרחשים

➤ ניתן להגדיר תנאים מעניינים



■ חסרונות:

➤ הזהרה, קיימת סכנה לעקוב אחר מימוש במקום אחרי המפרט

➤ רגישה לשינויים ב-DUV

➤ פרטים רבים מדי, מקשה לבניה ולתחזוקה

אימות קופסה אפורה

עבור אימות של קופסה אפורה, מספר מצומצם של מתקנים DUV מנוצלים בסביבה.

➤ יש גישה לרכיבים חשובים ויציבים, השאר נשמר "בחושך".

➤ משלב את היתרונות (אם נעשה את בדרך הנכונה) או החסרונות (אם נעשה בדרך לא נכונה) של קופסה שחורה ולבנה.

➤ בפועל, רוב סביבות אימות הן קופסה אפורה!

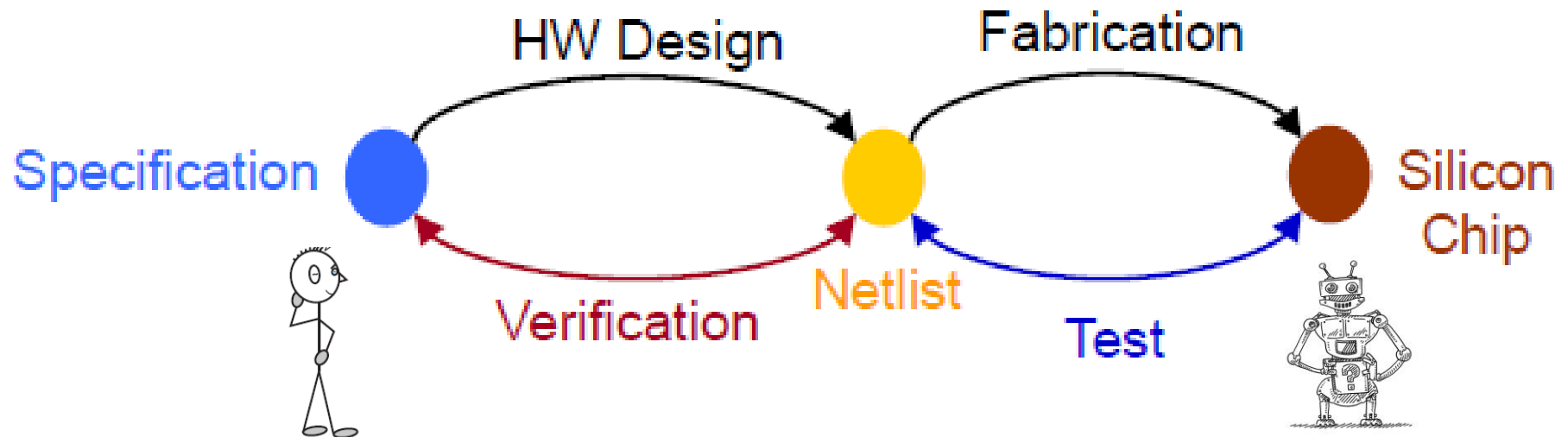
➤ חיזוי של תוצאות נכונות על תכנון בלתי אפשרי מבלי להבין את המבנה פנימי.

אימות וטסטים

מבולבלים לעיתים קרובות!

➤ מטרת הבדיקת טסטים היא להראות שהתכנון מיוצר כראוי.

➤ האימות נעשה כדי להבטיח שתכנון מקיים את הכוונה הפונקציונלית לפני הייצור!



הירארכית אימות

שבבים מורכבים של היום מחולקים ליחידות לוגיות

- החלוקה נקבעת בדרך כלל בזמן הגדרת מפרט רשמי ברמה גבוהה של התכנון
- החלוקה בדרך כלל נקבעת בעקבות הארכיטקטורה של המוצר
- השיטה הזאת נקראת תכנון הירארכי

אימות הירארכית

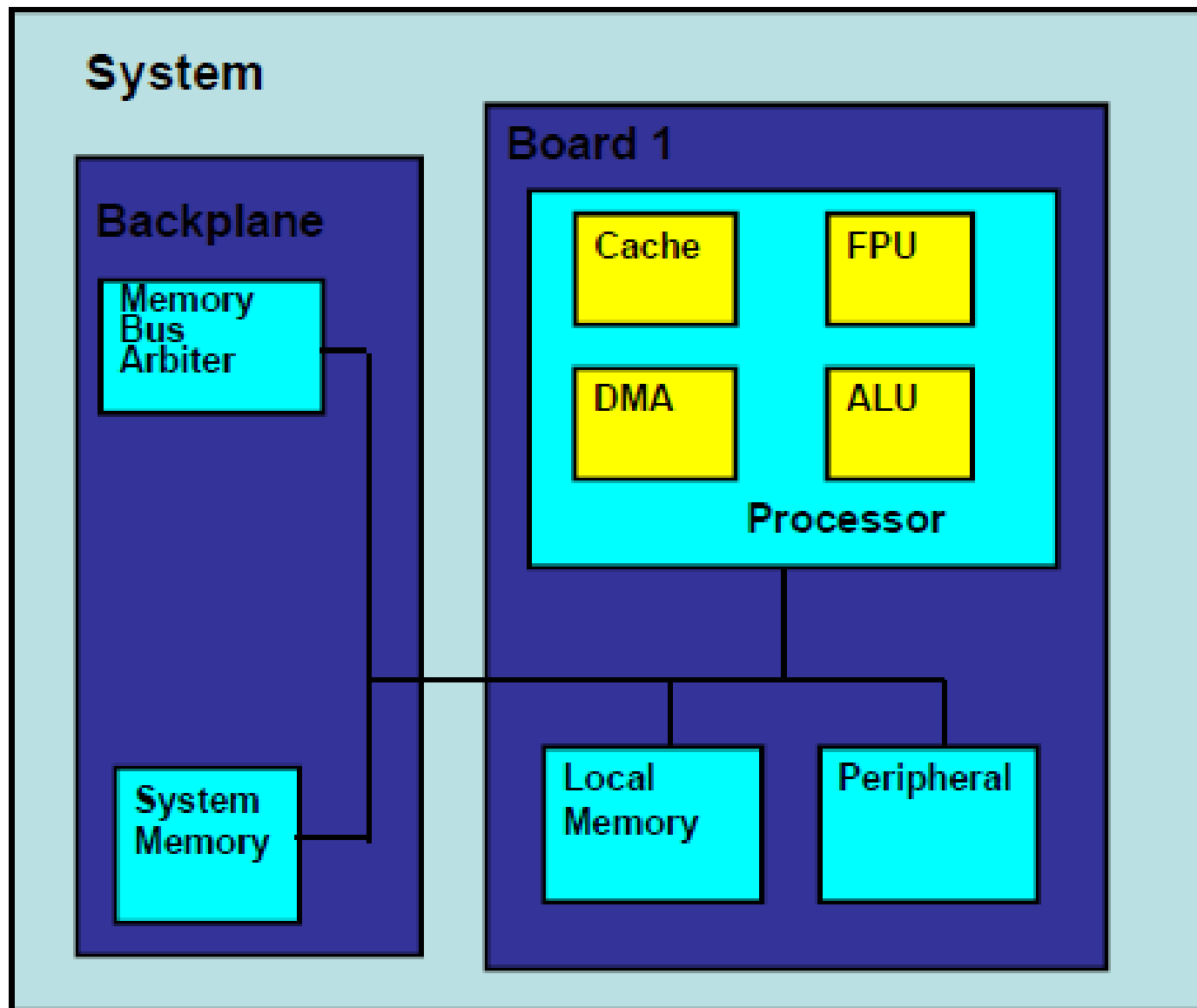
יתרונות ➤

- חלוקת התכנון ליחידות הניתנות לניהול
- מהנדס מרוכז על פונקציה אחת מוגדרת היבט של תכנון

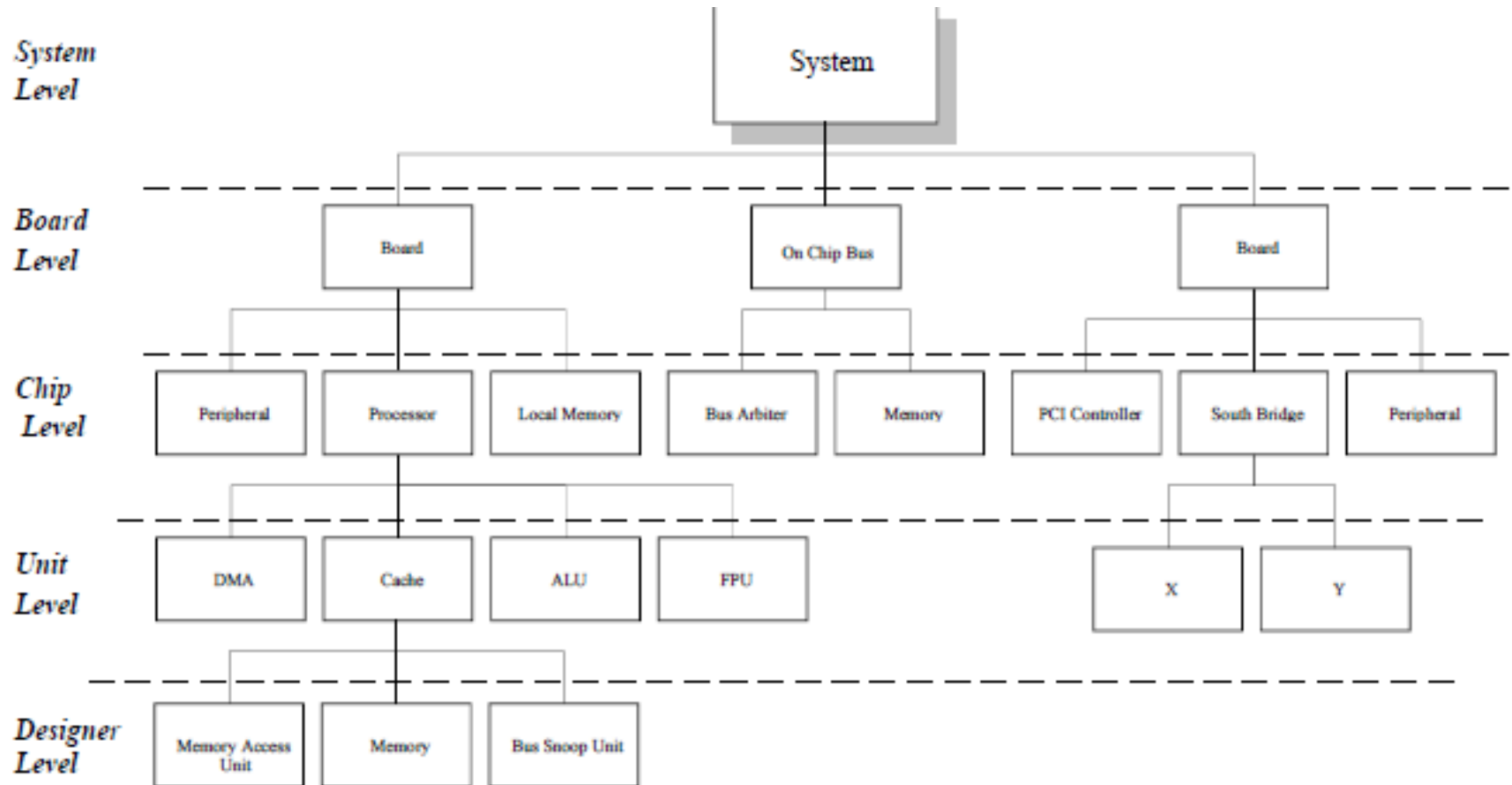
■ חסרונות

- ממשקים נוספים להגדרה / תכנון / אימות
- בעיות שילוב

מבנה הירארכי



מבנה הירארכי (המשך)



רמות אימות

➤ רמת הבלוק -- Block level

➤ רמת יחידה -- Unit level

➤ רמת הצ'יפ -- Chip level

➤ רמת הלוח -- Board level

➤ רמת המערכת -- System level

➤ אימות משיתוף חומרה / תוכנה



אימות ברמת הבלוק

- משמש עבור אימות של בלוקים בודדים
- בדרך כלל נעשה על ידי מהנדס האלקטרוניקה בעצמו
- המטרה העיקרית -בדיקת שפיות (Sanity checking)
וסרטיפיקציה של הבלוק הנתון
- כוללת גם בדיקות פשוטות של פונקציונליות בסיסית וגם
סביבות שלמות של האימות
- רמה נפוצה לאימות פורמלית

אימות ברמת יחידת

➤ קבוצה של בלוקים המיועדים לטפל
בפונקציה מסוימת מוגדרת היבט של המוצר
• למשל, בקר זיכרון

➤ בדרך כלל קיים מפרט רשמי של היחידה
• ממשק ופונקציונאליות יציבים יותר

➤ המטרה של המאמץ הראשון הרציני של אימות

➤ אימות מבוסס על סביבת אימות רצינית

אימות ברמת הצ'יפ

➤ אימות של קבוצה של יחידות שארוזה יחד לישות פיזית

➤ המטרות העיקריות של האימות

- נכונות של חיבור ושילוב של כל היחידות
- בדיקת פונקציות שלא ניתנת לביצוע ברמת היחידות

אימות ברמת הלוח/ מערכת

➤ מטרת ברמה זו של האימות היא לאשר:

- חיבורים
- אינטגרציה
- תכנון המערכת והלוח

➤ אימות מתמקדת באינטראקציות בין מרכיבי המערכת ולא
דווקא אל הפונקציונליות של כל מרכיב בניפרד

אימות משיתוף חומרה / תוכנה

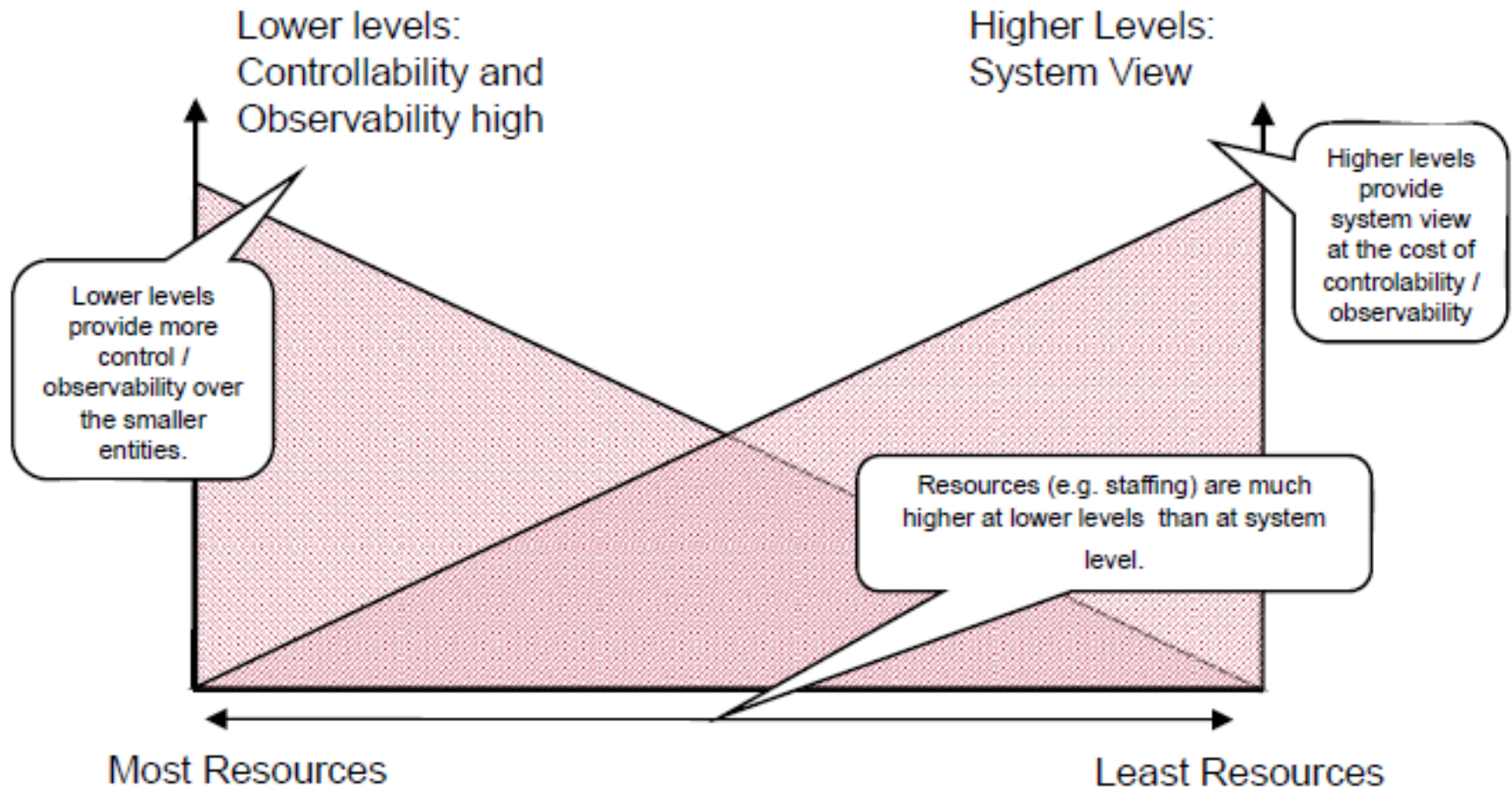
➤ משלב את החומרה ברמת מערכת עם התוכנה שרצה עליה

➤ משלבת טכניקות מן אימות החומרה ובדיקת התוכנה

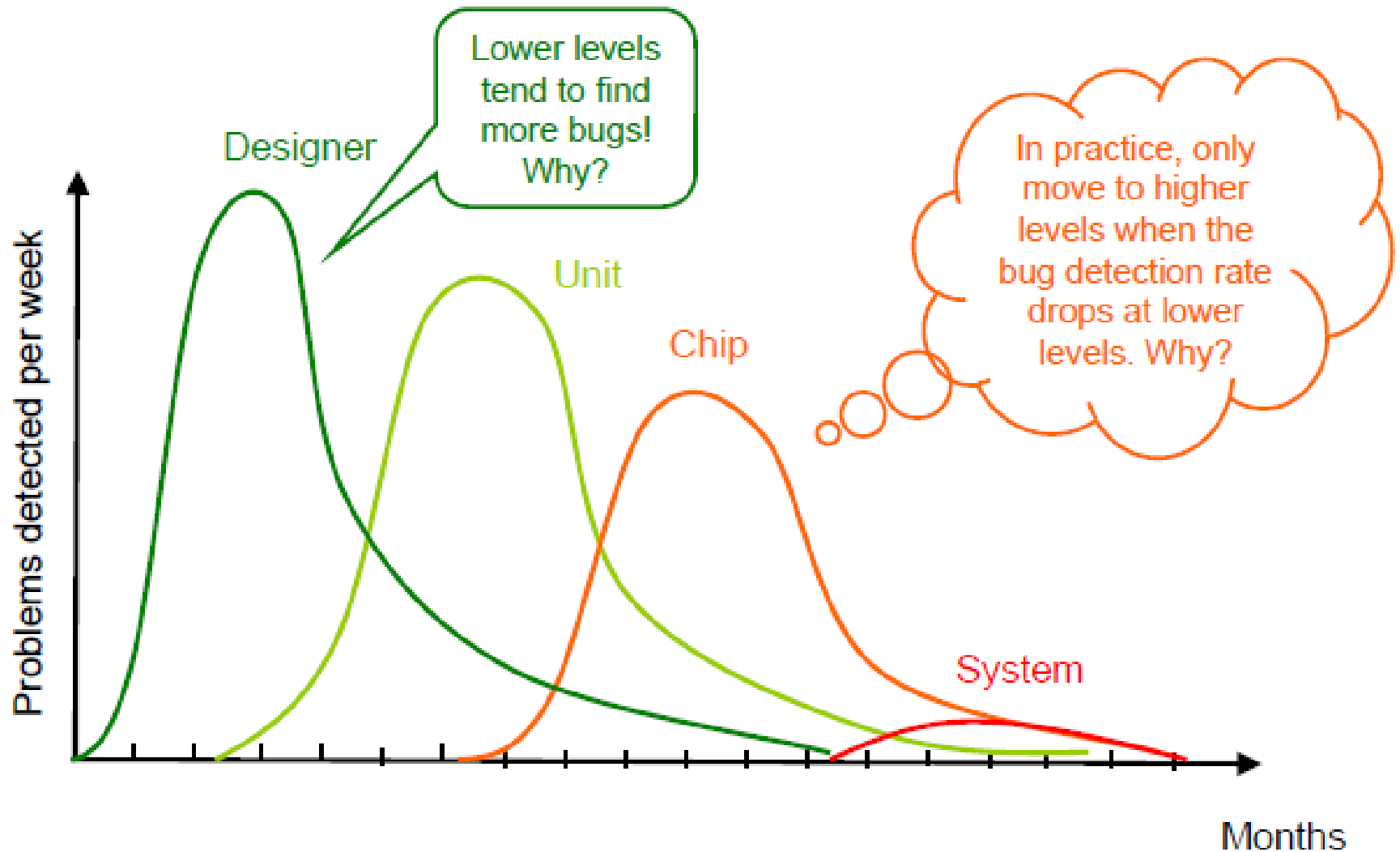
➤ שילוב זה יוצר בעיות רבות:

- טכניקות שונות אימות /בדיקה
- סוגים שונים של ביצוע
- מהירויות שונות

Trade-offs in Verification Hierarchy



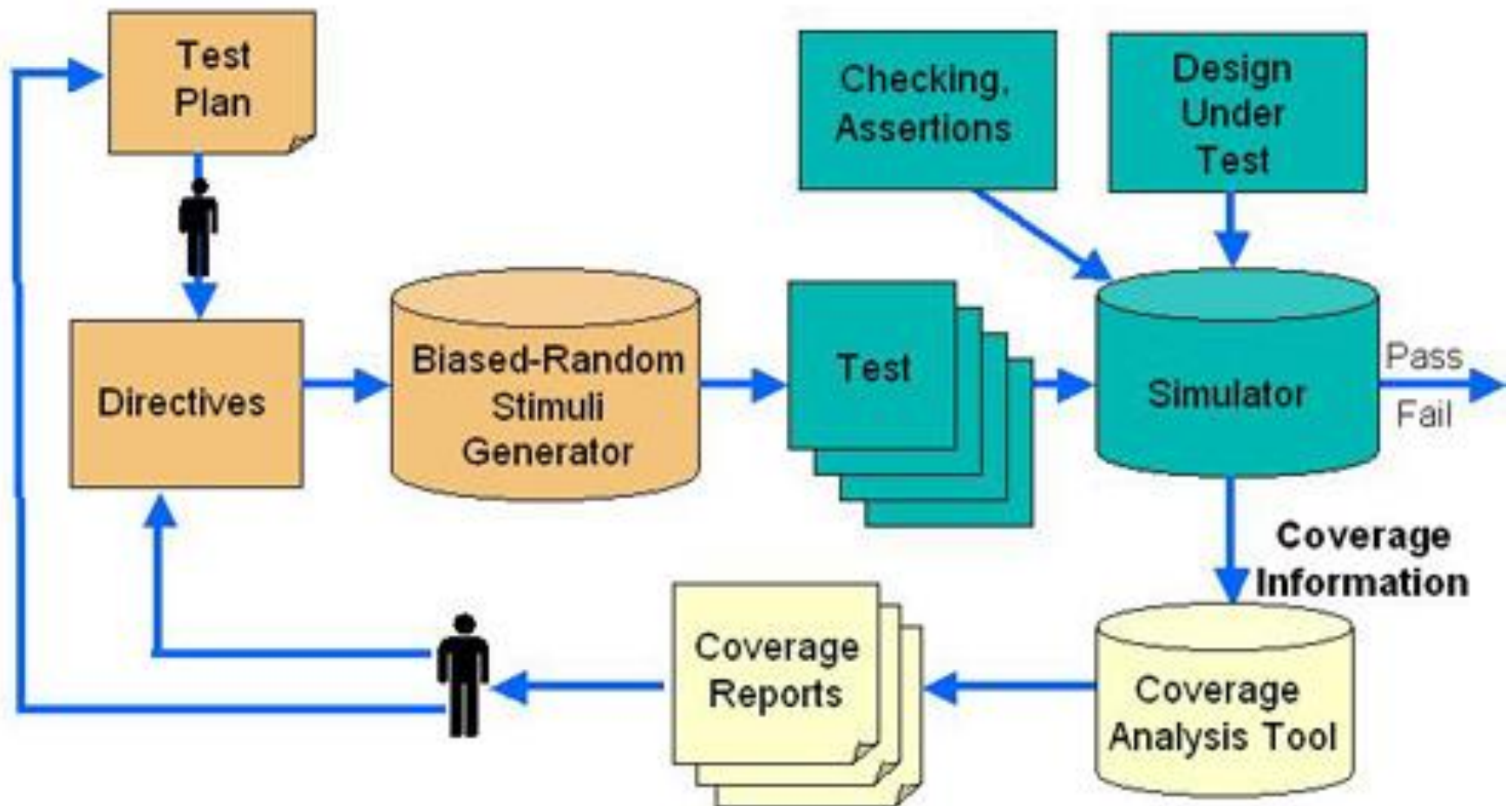
תפיסת באגים



אימות מבוססת סימולציה



אימות מבוסס סימולציה



אסטרטגיה של אימות

➤ אימות ניתן לחלק לשתי משימות נפרדות:

1. הרצת הרכיב – Controllability
2. בדיקת ההתנהגות שלו – Observability

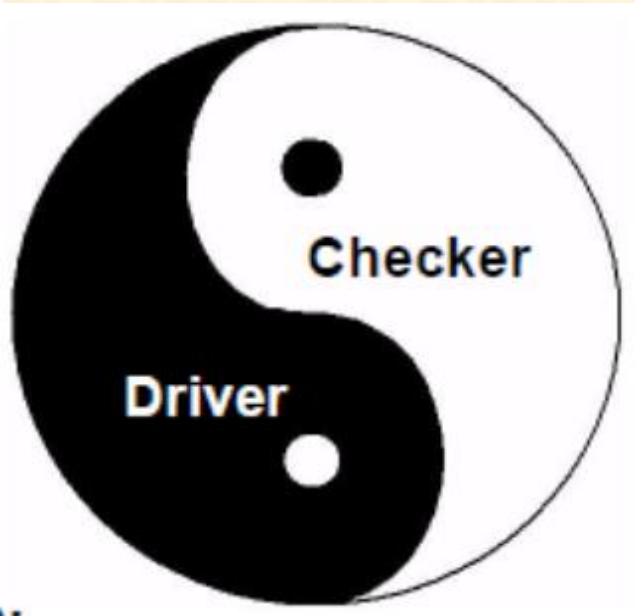
➤ שאלות בסיסיות שמהנדס האימות חייב לשאול:

1. האם אני מריץ את כל תרחישי הקלט האפשריים?
2. איך אני אדע מתי הכישלון התרחש?

Yin-Yang של אימות

➤ אנחנו לא יכולים למצוא באגים ללא יצירת תנאים לכישלון
• דריורים

➤ אנחנו לא יכולים למצוא באגים ללא גילוי התנהגות לא נכונה
• צקרים



אסטרטגיות בדיקות

➤ ישנם חמישה מקורות עיקריים של צקרים:

1. קלט ופלט של התכנון (מפרט)

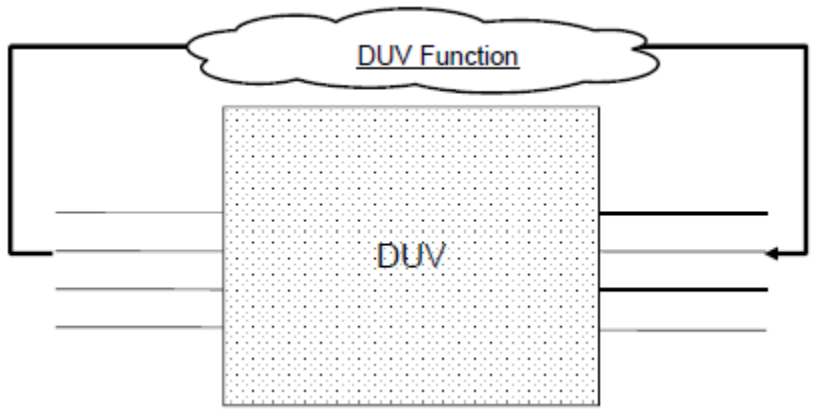
2. הארכיטקטורה של התכנון

3. המוקרו-ארכיטקטורה של

התכנון

4. יישום של התכנון

5. הסביבה של התכנון



➤ בדוק את אותות הפלט של DUV מבוסס על

- הקלט

- הבנה של המפרט של DUV

בדיקות מבוססות על קלט/פלט של DUV

➤ סוג בסיסי ביותר של בדיקות

- חייב להיות נוכחי, אלא אם כן אנו בטוחים כי זה סוג של בדיקות המכוסה על ידי סוגים אחרים של בדיקות
- הצקר לא צריך לשכפל את התכנון
- בדיקות קלות יותר מיישום של DUV
 - ❖ יכולות להשתמש ברמה גבוהה יותר של הפשטה
 - ❖ חייבות לאמת את התכנון במקום ליישם אותו

בדיקות המבוססות על ארכיטקטורה

➤ לבדיקות רבות יש שורשים בארכיטקטורת התכנון.
למשל:

✓ פקודה A חייבת להתבצע אחרי פקודה B

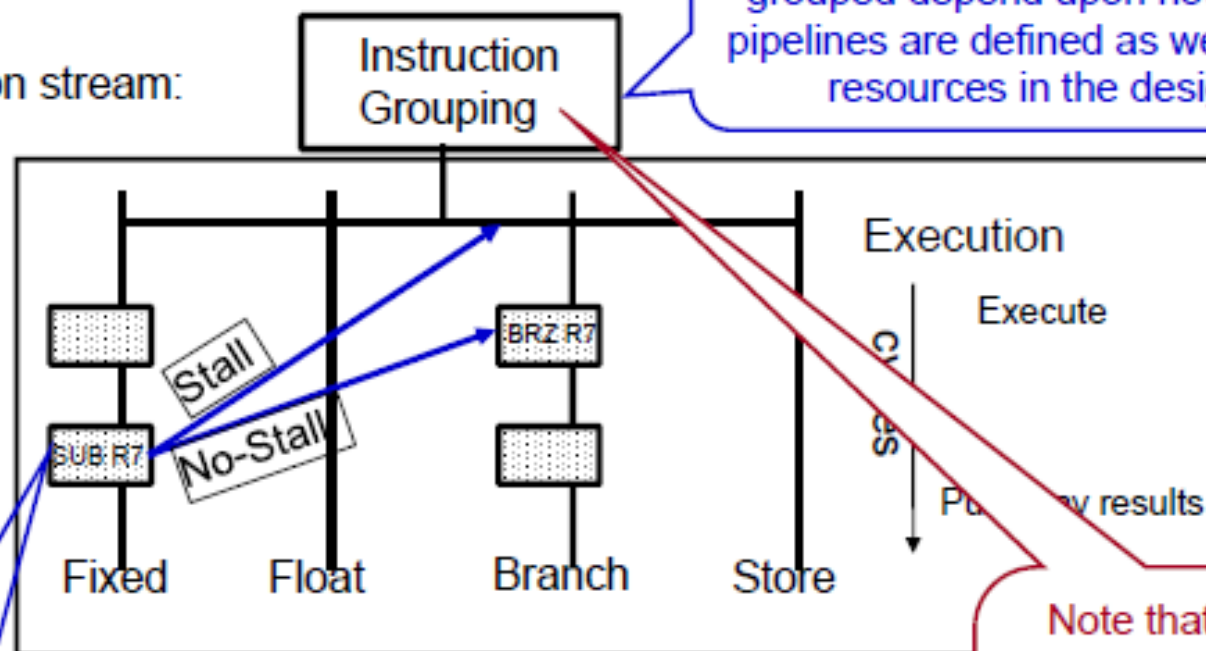
✓ תוצאת פקודה A חייבת להיות קלט של פקודה B

בדיקות המבוססות על מיקרו-ארכיטקטורת

Superscalar Pipeline

Example instruction stream:
SUB R7
BRZ R7

The rules on how instructions are grouped depend upon how many pipelines are defined as well as the resources in the design.

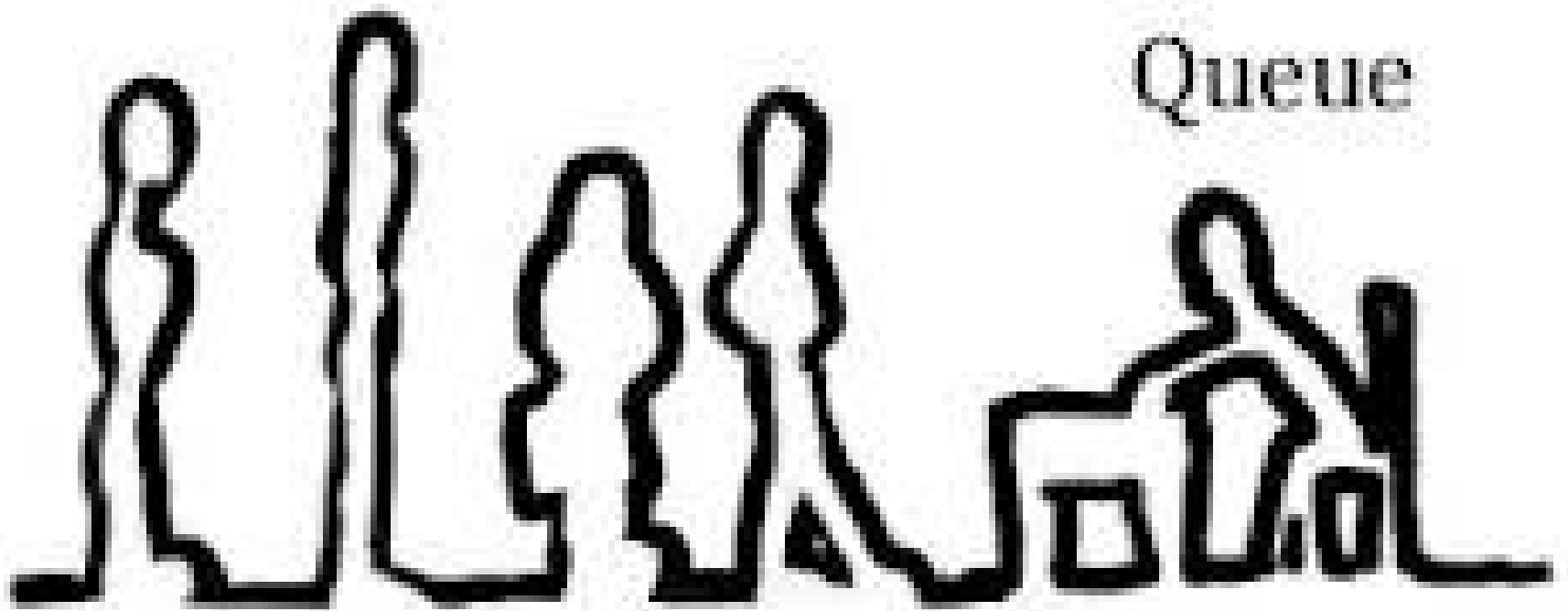


The ability or inability of on-the-fly results to feed prior stages of a pipeline will affect instruction grouping.

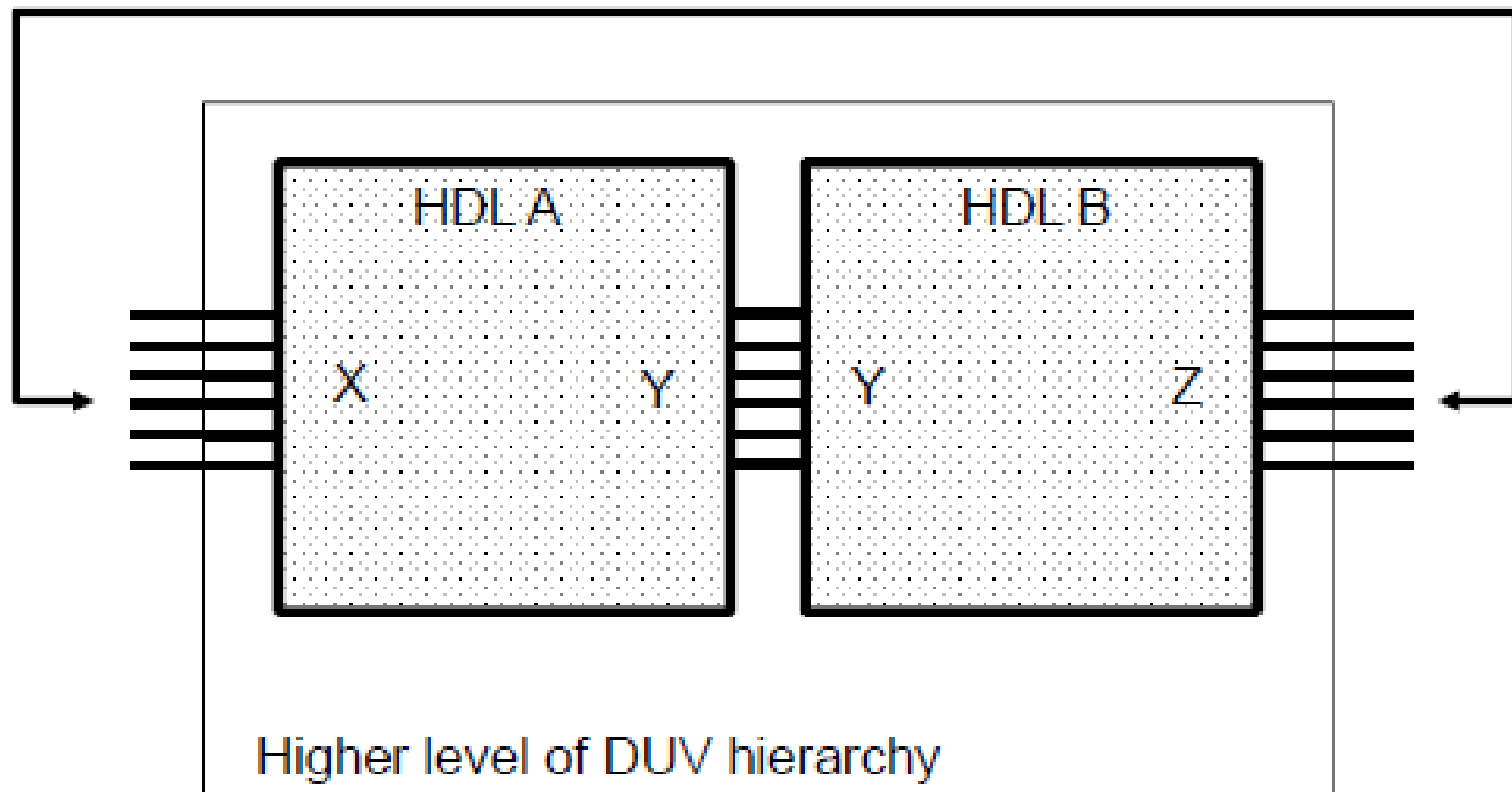
Note that if the DUV is the Instruction Grouping Unit then this is a design context example as well.

בדיקות המבוססות על יישום

בדוק פריטים הקשורים לפרטים ספציפיים של היישום
למשל: buffers מחזוריים עבור תורים



בדיקות מבוססות סביבת התכנון



בדיקות של קופסה שחורה (דוגמא)

Checker	Checker Source	Checker implementation
The design returns the correct data	Inputs and Outputs, Architecture	A fundamental check on the black box is that the returned data matches the sent data. The verification code must keep an independent copy of all DUV data in order to check the data outputs coming from the design.
Buffer overflow	Microarchitecture	The verification code must keep a count of how much data is in the design. This allows prediction and checking of the buf_full and buf_overflow outputs.
Data becomes valid at the right time	Microarchitecture	The design description stipulates that the driver may read data from the design the cycle after it sends it. Therefore, the verification team should write a checker to verify that the data is not valid too early/late and that it can be read the following cycle.
Check all outputs all of the time	Design context	The out_buf_data wires should never contain valid data unless the driver performed a read and there was data in the design. Similarly, the buf_full and buf_overflow wires should only be active during a full or overrun condition.